



Parallel and Distributed Computing Overview

(CS 3006)

Muhammad Aadil Ur Rehman

Department of Computer Science,
National University of Computer & Emerging Sciences,
Islamabad Campus

Credits: Dr. Muhammad Aleem, Dr. Muhammad Arshad Islam

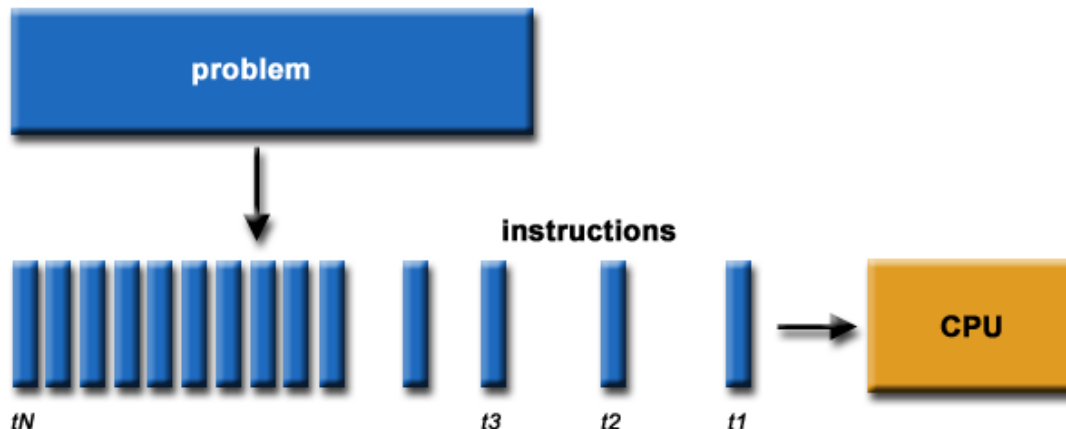


Parallel Thinking



Traditional Applications

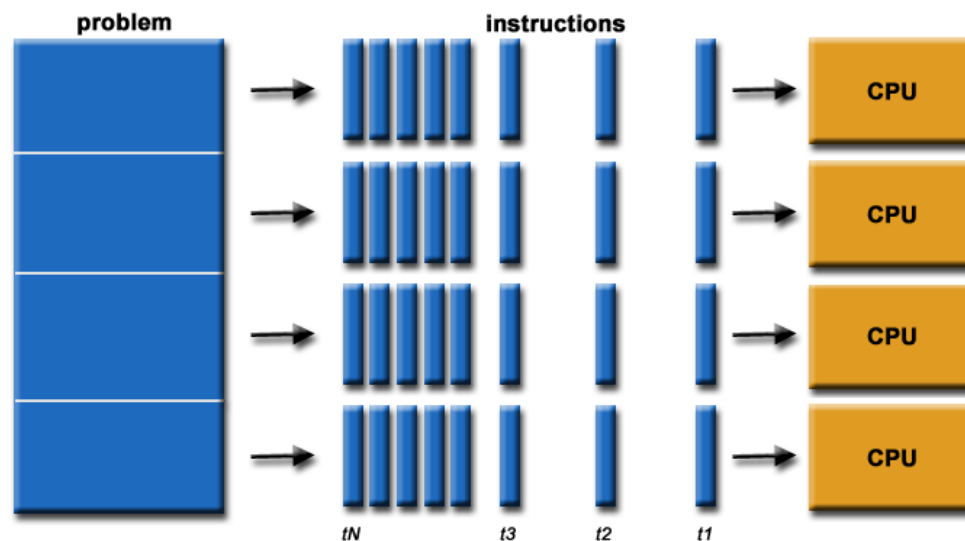
- Traditionally, software has been written for **serial computation**:
 - To be run on a **single computer** having a **single Central Processing Unit (CPU)**;
 - A **problem** is **broken** into a **discrete series** of instructions.
 - Instructions are executed **one after another**.





Traditional vs. Parallel Applications

- In the simplest sense, *parallel execution* is the **simultaneous** use of **multiple compute** resources to **solve a computational problem**
 - To be run using **multiple CPUs** (on same or different machines)
 - A problem is broken into discrete parts that can be solved **concurrently**
 - Each part is further broken down to a series of instructions





Recall Basic Terminologies

- **Task**

- A **logically discrete section of computational work**.
- A task is typically a **program** or **program-like set** of **instructions** that is **executed** by a **processor**.

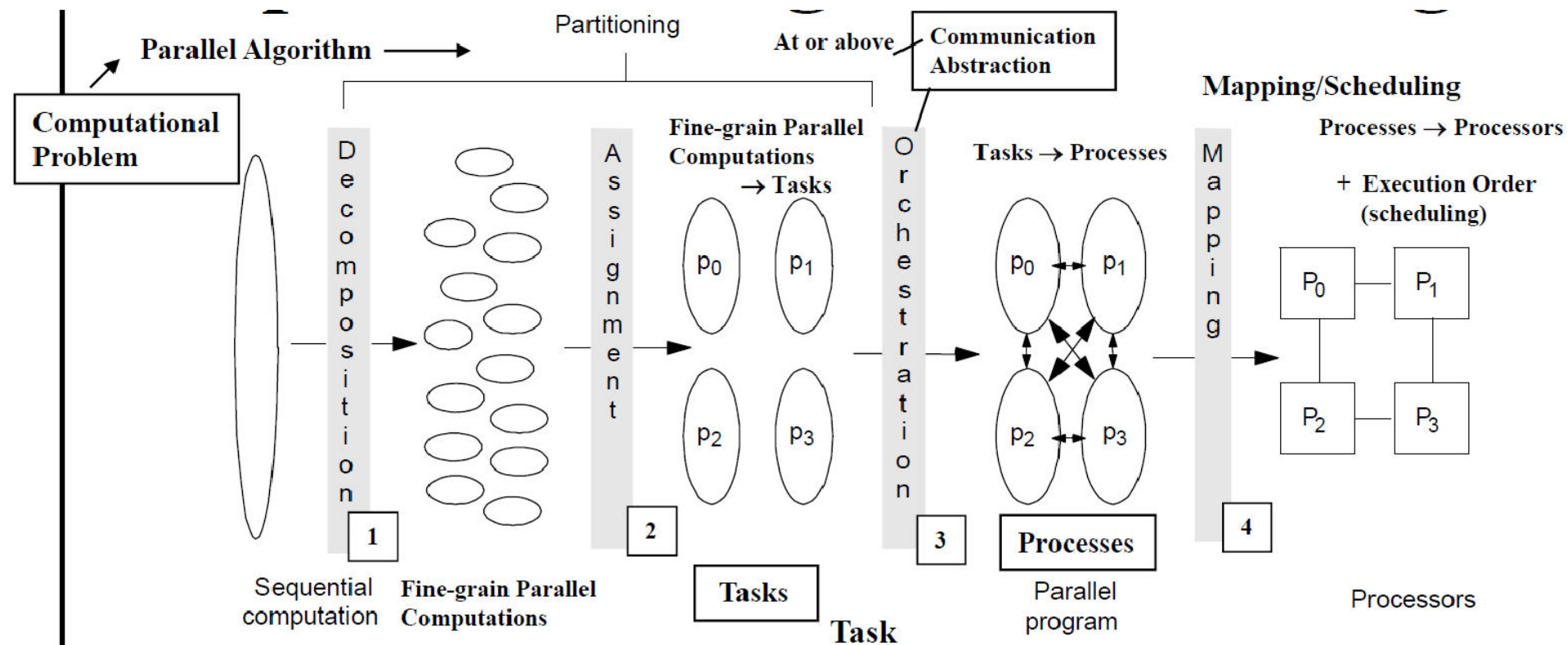
- **Parallel Task**

- A **task** that can be **executed** by **multiple processors** **safely** (*producing correct results*)

- **Serial Execution**

- **Execution** of a **program sequentially**, one statement at a time using one processor.

4 Steps in Creating Parallel Programs



- **Decomposition** of computation in tasks
- **Assignment** of tasks to processes
- **Orchestration** of data access, comm, synch.
- **Mapping** processes to processors



Parallelization Strategy

- 1. Problem Understanding**
- 2. Partitioning/Decomposition**
- 3. Assignment**
- 4. Orchestration**
- 5. Mapping**



Parallelization Strategy

- 1. Problem Understanding**
2. Partitioning/Decomposition
3. Assignment
4. Orchestration
5. Mapping



1. Problem Understanding

- The **first step** in developing parallel application is to **Understand the Problem** (that you wish to solve)
- Starting with a **serial program** [?] **Understanding** the **existing code**
- The **first decision** [?] Whether or **not** the **problem** is **suitable** for parallelization?
 - *Code Dependencies*
 - *Communication*
 - *Synchronization*



1. (a) Identify the Programs hot-spots

- **Know where most of the real work is being done.**
*(The majority of **scientific** and **technical** programs usually **accomplish most of their work in a few places.**)*
- **Profilers** and **performance analysis tools** can help here
- **Focus** on **parallelizing** the **hotspots** and **ignore those sections** of the program that **account for little CPU usage.**



1. (a) Identify the Programs hot-spots

- Identify **areas** that are **unjustly slow**?
 - Example: **I/O** is usually something that slows a program
- **Potential Solutions:**
 - **Restructure the program**
 - **Use a different algorithm**
 - **Overlap the communication with computation**



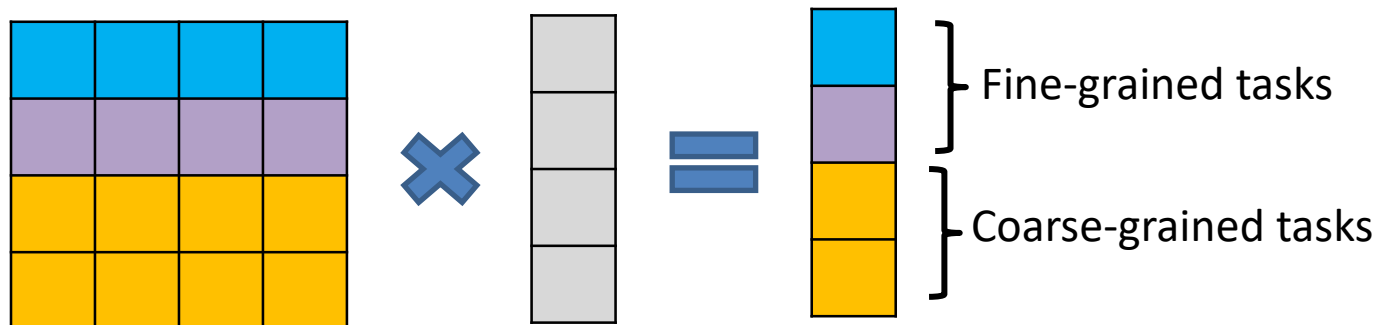
Parallelization Strategy

1. Problem Understanding
- 2. Partitioning/Decomposition**
3. Assignment
4. Orchestration
5. Mapping



2. Partitioning/Decomposition

- **Divide work** into discrete "**chunks**"
- Broken **chunks** can be **assigned** to **specific tasks**
- **Tasks** could be **executed concurrently**
- **Decomposition**: (1) **Domain decomposition**
(2) **Functional decomposition**
- **Decomposition Granularity**:
 - **Fine-grain**: large number of small tasks
 - **Coarse-grain**: small number of large tasks





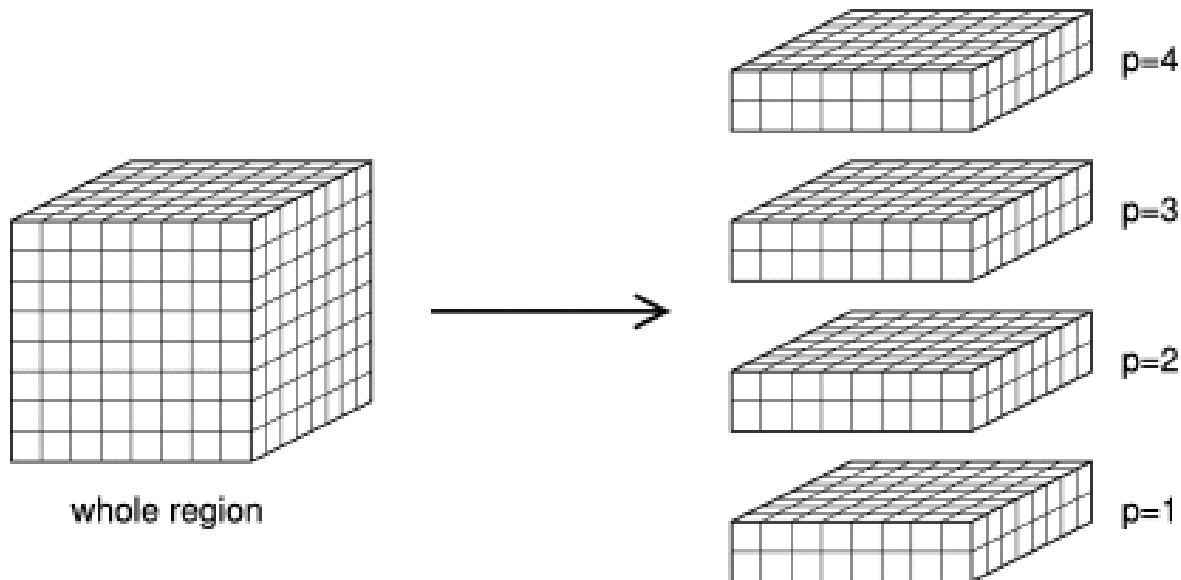
Granularity

- **Grain of parallelism:** How big are the units?
 - Bits, Instructions, Blocks, Loop iterations, Procedures, ...



2. (a) Domain Decomposition

- The **data** associated with a problem is **decomposed**
- Each **parallel task** then **works** on a **portion of the data**



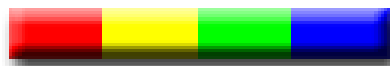


2. (a) Domain Decomposition

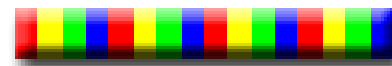
1D Data Decomposition:

- **Block:** same sized large blocks (typically == *number of parallel tasks*) - Pros & Cons?
- **Cyclic:** small-sized many blocks (typically >> *number of parallel tasks*) - Pros & Cons?

1D



BLOCK



CYCLIC



2. (a) Domain Decomposition

2D Data Decomposition:

- **Block (row*,column*)**: same sized large blocks (*row or column*) (typically == *number of parallel tasks*) - Pros & Cons?
- **Block (row,column)**: small sized sub-blocks (typically == *number of parallel tasks*) - Pros & Cons?



BLOCK, *



*, BLOCK



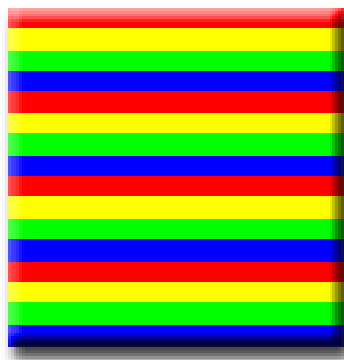
BLOCK, BLOCK



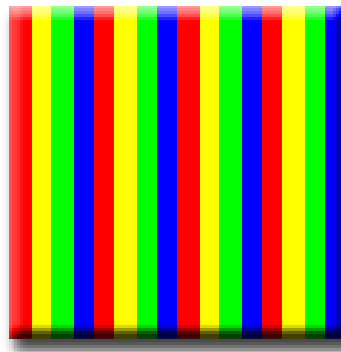
2. (a) Domain Decomposition

2D Data Decomposition:

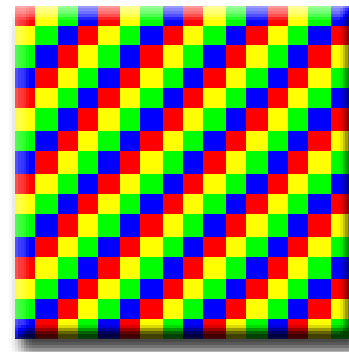
- **Cyclic (row*,column*)**: small-sized many blocks (row or column) (typically $\gg$ number of parallel tasks) - Pros & Cons?
- **Cyclic (row,coulmn)**: small-sized many sub-blocks (typically $\gg$ number of parallel tasks) – Pros & Cons?



CYCLIC, *



*, CYCLIC



CYCLIC, CYCLIC



2. (b) Functional Decomposition

- **Focus** is on the **computation** that is to be performed
- The **problem** is **decomposed** according to the **work** (instructions) that must be done
- Each **sub-task** performs a **portion** of the **overall work**

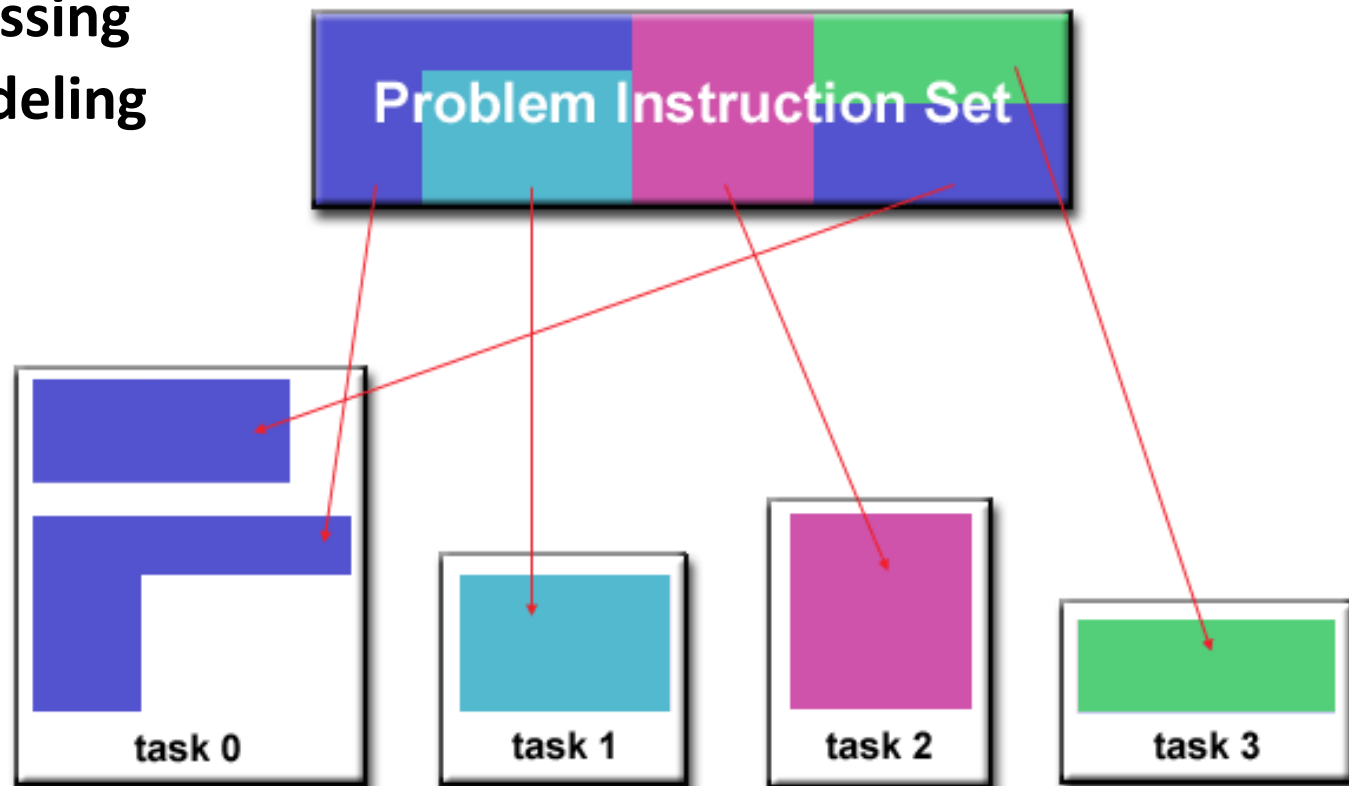


2. (b) Functional Decomposition

- **Functional decomposition** useful to the problems that can be **split into different tasks**

Examples:

- Ecosystem Modeling
- Signal Processing
- Climate Modeling





Parallelization Strategy

1. Problem Understanding
2. Partitioning/Decomposition
- 3. Assignment**
4. Orchestration
5. Mapping



3. Assignment

- **Composing** “Fine-grained” parallel computations into **processes** or **coarse-grained tasks**
- **Considerations** while **composing tasks**: *Load-balance, Uniform communication, ease of Synchronization*

	Fine-Grain Parallelism	Coarse-Grain Parallelism
Computation/Communication ratio	Low	High
Computational work between communication stages	Small	Large
Communication/Synchronization overhead	High	Low
Load balance efficiency	High	Low



Parallelization Strategy

1. Problem Understanding
2. Partitioning/Decomposition
3. Assignment
- 4. Orchestration**
- 5. Mapping**



4-5. Orchestration & Mapping

- Following **aspects** should be **considered** before **actual mapping (Scheduling)**:
 - a) Inter-task communication
 - b) Synchronization among tasks
 - c) Data locality aspect
 - d) Other system related considerations (NUMA, etc.).



Communication



Who Needs Communications?

- **No Communication Required:**
 - **Some problems** can be **decomposed** and **executed** in **parallel** with **virtually no need** for tasks **to share data**:
 - Imagine an **image processing** operation where **every pixel** in a **black** and **white** image **needs** to have its **color reversed**
 - **These types of problems** are often **called Embarrassingly Parallel** because they are so *straight-forward*.
 - **Very little or No inter-task communication** is required



Who Needs Communications?

- **Communications Required**

- Most **parallel applications** are **not quite so simple**,
- They **do require tasks to share data** with **each other**
 - **For example:** a 3-D **heat diffusion** problem **requires a task** to know the **temperatures calculated** by the **tasks** that have **neighbouring data**
 - **Changes to neighbouring data** has a **direct effect** on that **task's data**



Communication Cost

- **Inter-task communication** virtually **always** implies **overhead**
- **CPU time** and **resources** are used to **package** and **transmit data**
- **Communications** **mostly** **require** **some** **synchronization** between **tasks** ☐ **Results in** **waiting time**
- **Communication** can **saturate** the **available network bandwidth**, **further reducing performance**



Latency & Bandwidth

- Latency is the time it takes to send a message from point A to point B
- Bandwidth is the amount of data that can be communicated per unit of time
- Sending many small messages can cause a higher overall latency
- Efficient approach is to package small messages into a larger message
 - To maximize bandwidth usage & Reduce Overhead



Visibility of communications

- In Distributed memory model, **communications** are **explicit** and **under the control** of a **programmer**
- In Shared memory mode **communications** are implicit (transparently to the programmer)



Communication Patterns

A. Synchronous communication: **require** some type of "handshaking" between tasks

- **Blocking communications** (since other task must wait until the communications have completed)

A. Asynchronous communications: allow **tasks to transfer data independently** from one another

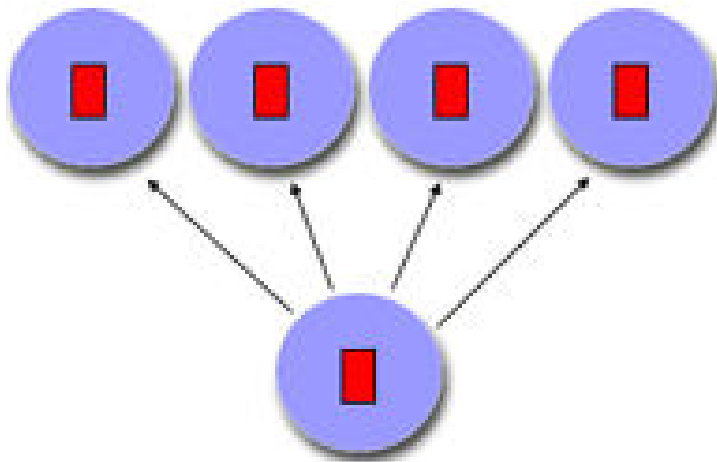
- **non-blocking** communications
- **Interleaving computation with communication** is the single greatest benefit



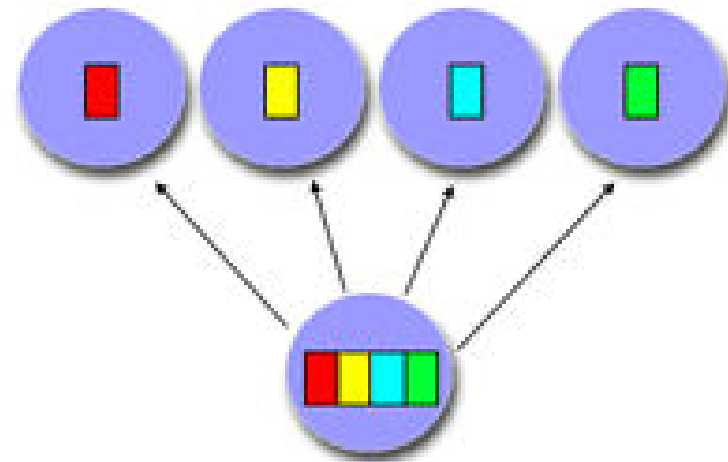
Scope of Communication

- **Knowing** which tasks must **communicate with each other** is **critical** during the design stage of a parallel code
 1. **Point-to-point** - involves **two tasks** with **one task** acting as the **sender/producer** of data, and the **other** acting as the **receiver/consumer**.
 1. **Collective** - involves **data sharing** between **more than two tasks**, which are often specified as **being members** in a **common group**, or collective.

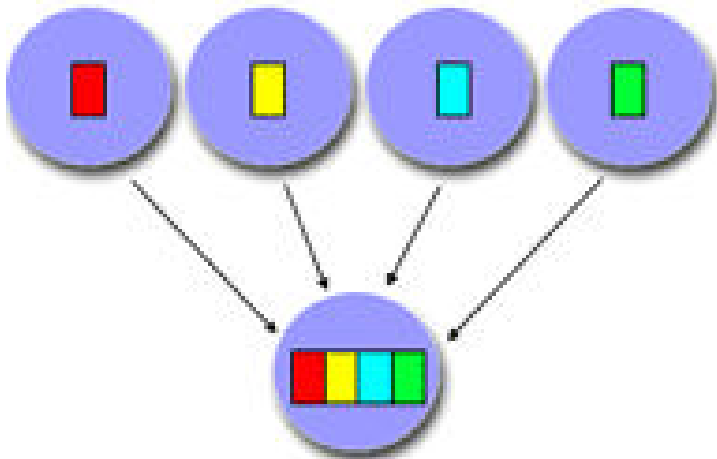
Collective Communications



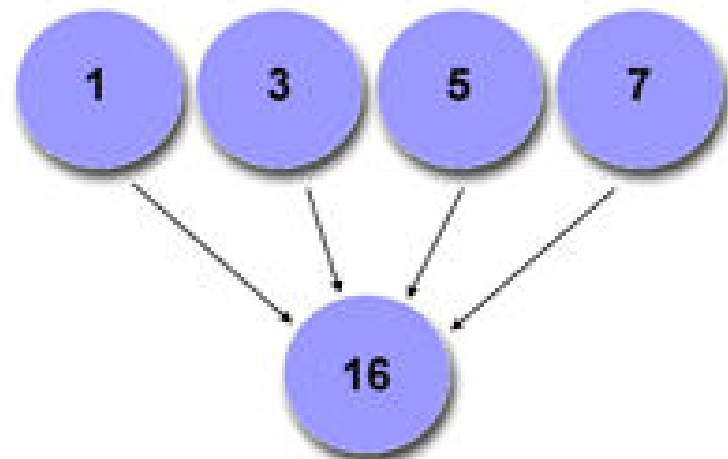
broadcast



scatter



gather



reduction



Synchronization



Synchronization

- **Synchronization:** mechanism for coordination and sequencing of parallel tasks
- Significant factor in program's performance
- Often requires "serialization" of program segments



Synchronization Types

1. Barrier

- Usually **all tasks are involved**
- Each **task performs** its **work** until it **reaches the barrier**. It then **stops**, or "**blocks**"
- When the **last task reaches the barrier**, **all tasks resume**



Synchronization Types

2. Lock / semaphore

- Can involve any number of tasks
- Typically used to serialize (protect) access to global data/code. Only one task at a time may use (own) the lock / semaphore / flag.
- The first task to acquire the lock "sets" it
- Other tasks can attempt to acquire the lock but must wait until the task that owns the lock releases it
- Can be blocking or non-blocking



Synchronization Types

3. Synchronous communication Operations

- **Involves** only **those tasks** executing a **communication operation**
- **Before** **initiating** the **communication**, **acknowledgement** must be **received** (*OK to send*)



Any Questions?